TALENT NURTURES
NURTURING THE GEN NEXT!!

NCC Global Assignment

NCC education

Qualification: L5DC-BM

Unit Name: Introduction to Data Science and Big Data

Name of Student: Mhd Laith Alkhateeb

Student NCC ID: 213142

Address: SFO0003, Compass Building, Al Hamra Industrial Zone,

Ras Al Khaima, UAE

Contact No: +971-4-2628591 www.tnedu.org

Assignment Marking Sheet

Awarding Great British Qualifications

Centre Name	Talent Nurtures Educational Institute
Candidate Name	Mhd Laith Alkhateeb
NCC Education Candidate ID No.	213142
Examination Date	15/8/2025
Examination Unit Title	Google Local Research Team

Questions answered by the candidate:

Please insert the question numbers you have answered in the table below.

Question number attempted (To be completed by Candidate)	*Marks Awarded (To be completed by 1st marker)*	*Marks Awarded (To be completed by 2nd marker)*
TOTAL MARKS		

First Marker Name & Signature: Muhammad Rafiq

Second Marker Name & Signature: Naseem Gul 

Statement. and .Confirmation .of .Own .Work

Awarding Great British Qualifications

A signed copy of this form must be submitted with every assignment. If the statement is missing you work may not be marked. For Level 7 assignments, please use the separate Level 7 Candidate statement of own work from instead of this one.

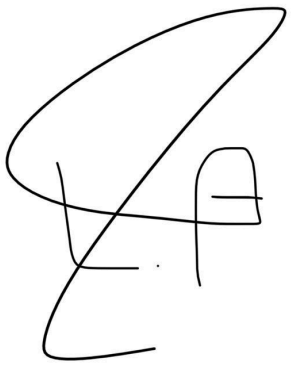
I confirm the following details:

Candidate Name:	Mhd Laith Alkhateeb
Candidate ID Number:	213142
Qualification:	L5DC-BM
Unit:	Introduction to Data Science and Big Data
Centre:	Talent Nurtures Educational Institute
Word Count:	2390

I have read and understood both NCC Education's Academic Misconduct Policy and the Referencing and Bibliographies document. To the best of my knowledge my work has been accurately referenced and all sources cited correctly.

I confirm that I have not exceeded the stipulated word limit by more than 10%.

I confirm that this is my own work and that I have not colluded or plagiarised any part of it.

Candidate Signature:	
Date:	15/8/2025

Mhd Laith Alkhateeb ID No: 213142 Word Count: 2390

Google Local Research Team

Mhd Laith Alkhateeb ID No: 213142 Word Count: 2390

Scenario

You are a member of the Research and Development team of Google Local, i.e., a Google Map service that allows users to write reviews and add ratings of places they have visited. Your team has collected three large-scale datasets, namely 1) Users, 2) Reviews and 3) Places. Please see Google Local Folder that details the attributes and relationships of the three datasets. Note that the datasets contain real-world data which is publicly available.

Mhd Laith Alkhateeb ID No: 213142 Word Count: 2390

Task 1: Data Pre-processing

This task illustrates the Data Pre-processing process which includes preparing the three (3) files for analysis by performing the data cleaning.

Data Pre-processing is a procedure where Data gets prepared for analysis. This procedure involves cleaning the data by performing techniques such as converting the unstructured data to structured, spell checking, removing contractions, and tokenizing. It is like a student organizing his/her desk before studying.

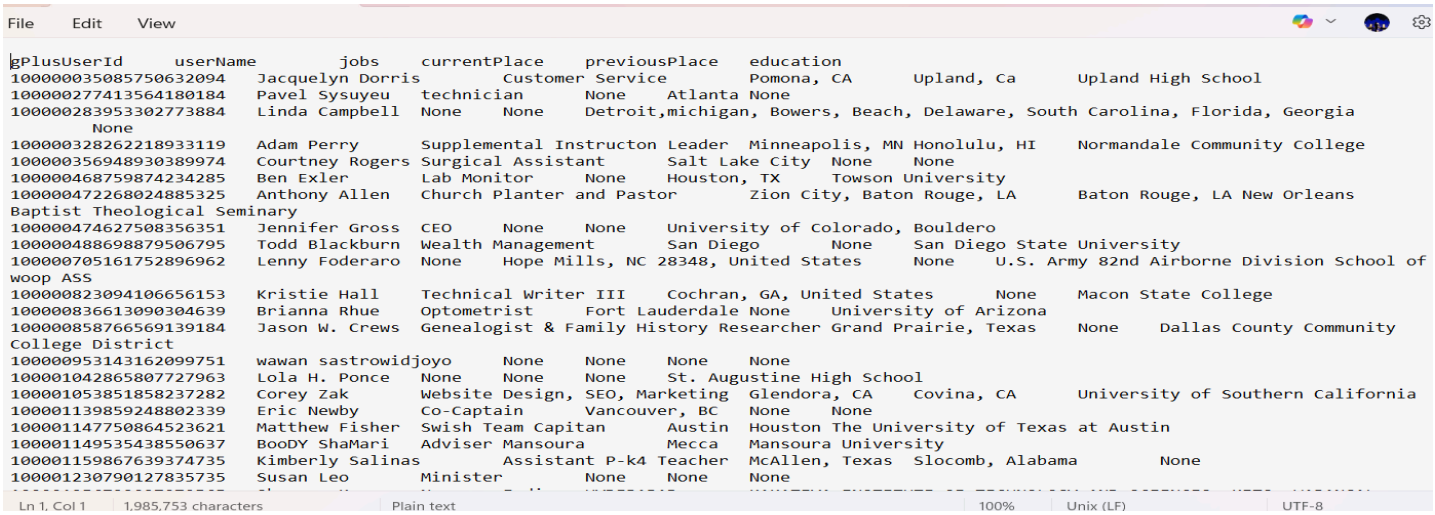
The tool used in the data cleaning procedure is Python with Jupyter Notebook and pandas as it can read various types of files and contains libraries for most of the data cleaning methods.

(dataquest.io , Using Excel with Python and Pandas, Dec13,2024) (packtpub.com , 10 reasons why scientists love Jupyter notebooks, apr4,2018)

Uploading files for users dataset:

```
ParserError: Error tokenizing data. C error: Expected 1 fields in line 71, saw 2
```

When uploading the users dataset file, a ParserError was seen appearing because pandas initially expected a comma separated file.



```
File    Edit    View

gPlusUserId     userName        jobs    currentPlace    previousPlace   education
100000035085750632094   Jacquelyn Dorris        Customer Service        Pomona, CA      Upland, Ca      Upland High School
100000277413564180184   Pavel Sysuyeu   technician      None    Atlanta None
100000283953302773884   Linda Campbell  None    None    Detroit,michigan, Bowers, Beach, Delaware, South Carolina, Florida, Georgia
        None
100000328262218933119   Adam Perry      Supplemental Instructon Leader  Minneapolis, MN Honolulu, HI    Normandale Community College
100000356948930389974   Courtney Rogers Surgical Assistant      Salt Lake City  None    None
100000468759874234285   Ben Exler       Lab Monitor     None    Houston, TX     Towson University
100000472268024885325   Anthony Allen   Church Planter and Pastor       Zion City, Baton Rouge, LA      Baton Rouge, LA New Orleans
Baptist Theological Seminary
100000474627508356351   Jennifer Gross  CEO     None    None    University of Colorado, Bouldero
100000488698879506795   Todd Blackburn  Wealth Management       San Diego       None    San Diego State University
100000705161752896962   Lenny Foderaro  None    Hope Mills, NC 28348, United States     None    U.S. Army 82nd Airborne Division School of
woop ASS
100000823094106656153   Kristie Hall    Technical Writer III    Cochran, GA, United States      None    Macon State College
100000836613090304639   Brianna Rhue    Optometrist     Fort Lauderdale None    University of Arizona
100000858766569139184   Jason W. Crews  Genealogist & Family History Researcher Grand Prairie, Texas    None    Dallas County Community
College District
100000953143162099751   wawan sastrowidjoyo     None    None    None    None
100001042865807727963   Lola H. Ponce   None    None    None    St. Augustine High School
100001053851858237282   Corey Zak       Website Design, SEO, Marketing  Glendora, CA    Covina, CA      University of Southern California
100001139859248802339   Eric Newby      Co-Captain      Vancouver, BC   None    None
100001147750864523621   Matthew Fisher  Swish Team Capitan      Austin  Houston The University of Texas at Austin
100001149535438550637   BooDY ShaMari   Adviser Mansoura        Mecca   Mansoura University
100001159867639374735   Kimberly Salinas        Assistant P-k4 Teacher  McAllen, Texas  Slocomb, Alabama        None
100001230790127835735   Susan Leo       Minister        None    None    None

Ln 1, Col 1     1,985,753 characters    Plain text      100%    Unix (LF)       UTF-8
```

The dataset file was opened in Notepad in order to gain a clear understanding about the file and how the columns are separated. Moreover, the results shown that the columns were separated by tabs. Therefore, the file was reloaded with (delimiter="\t") to inform. the pandas that the file is tab separated not comma separated.

```python
import pandas as pd

users = pd.read_csv(
    r"C:/Users/laith/OneDrive/Laith Studying/L5DC-BM Global Assignments/Introduction to data science and big data/I
    delimiter="\t"
)

print(users.shape)
users.head()
users.dtypes
```

```
(21858, 6)

]: gPlusUserId      object
   userName         object
   jobs             object
   currentPlace     object
   previousPlace    object
   education        object
   dtype: object
```

After detecting and fixing the error, the users dataset file was loaded successfully with all 6 columns.

Breakingdown Data for users dataset:

```
[5]: users.head()
     users.info()
     users.isnull().sum()
     users.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21858 entries, 0 to 21857
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   gPlusUserId    21858 non-null  object
 1   userName       21858 non-null  object
 2   jobs           17952 non-null  object
 3   currentPlace   15343 non-null  object
 4   previousPlace  8478 non-null   object
 5   education      15585 non-null  object
dtypes: object(6)
memory usage: 1.0+ MB
```

[5]:

	gPlusUserId	userName	jobs	currentPlace	previousPlace	education
count	21858	21858	17952	15343	8478	15585
unique	21858	21705	10329	9174	6179	9648
top	100000035085750632094	David Rodriguez	Owner	London	London	University of Phoenix
freq	1	4	751	124	41	80

In order to proceed to the next stage(which is data cleaning), the data must be broken down to check the structure of the data, the types of the data, size of the data, and potential issues like missing values. Therefore, an inspection command was run .

Following the breakdown procedure, the results shown as follows:

1- the data has been successfully loaded with 21,858 rows and 6 columns

2- No missing values, which can simplify the cleaning process

3-There are some repetitions, especially in the jobs column as the job type (Owner) was mentioned 751.

Removing Duplicates for users dataset:

```
print(users.shape)
users.drop_duplicates(subset=['gPlusUserId'], inplace=True)
print(users.shape)
```

```
(21858, 6)
(21858, 6)
```

Removing duplicates is the next step taken after successfully breaking down the dataset. A duplicate command (drop_duplicates) was initiated to find out any duplicated values. The results showed that the number of rows did not change. Which is a sign that there are no duplicates to remove.

Structure Conversion for users dataset:

```
[40]: users['jobs'] = users['jobs'].apply(lambda x: x.split(',') if isinstance(x, str) and pd.notna(x) else [])
users['jobs'] = users['jobs'].apply(lambda x: [job.strip() for job in x] if isinstance(x, list) else x)
print(users['jobs'].head(10))
```

```
0                 [Customer Service]
1                       [technician]
2                                 []
3    [Supplemental Instructon Leader]
4                [Surgical Assistant]
5                       [Lab Monitor]
6         [Church Planter and Pastor]
7                               [CEO]
8                 [Wealth Management]
9                                 []
Name: jobs, dtype: object
```

```
]: print(users['currentPlace'].head(10))
   print(users['education'].head(10))
```

```
0                                Pomona, CA
1                                      NaN
2                                      NaN
3                          Minneapolis, MN
4                           Salt Lake City
5                                      NaN
6              Zion City, Baton Rouge, LA
7                                      NaN
8                                San Diego
9    Hope Mills, NC 28348, United States
Name: currentPlace, dtype: object
0                                    Upland High School
1                                                   NaN
2                                                   NaN
3                          Normandale Community College
4                                                   NaN
5                                       Towson University
6              New Orleans Baptist Theological Seminary
7                       University of Colorado, Bouldero
8                             San Diego State University
9    U.S. Army 82nd Airborne Division School of woo...
Name: education, dtype: object
```

```
]: users[['city', 'state']] = users['currentPlace'].str.split(',', n=1, expand=True)
```

```
6]: print(users[['city', 'state']].head(10))
```

	city	state
0	Pomona	CA
1	Unknown	Unknown
2	Unknown	Unknown
3	Minneapolis	MN
4	Salt Lake City	Unknown
5	Unknown	Unknown
6	Zion City	Baton Rouge, LA
7	Unknown	Unknown
8	San Diego	Unknown
9	Hope Mills	NC 28348, United States

```
7]: users[['edu_part1', 'edu_part2']] = users['education'].str.split(',', n=1, expand=True)
```

```
3]: users['edu_part1'] = users['edu_part1'].fillna('Unknown')
    users['edu_part2'] = users['edu_part2'].fillna('Unknown')
```

```
9]: print(users[['edu_part1', 'edu_part2']].head(10))
```

```
                                        edu_part1  edu_part2
0                              Upland High School    Unknown
1                                         Unknown    Unknown
2                                         Unknown    Unknown
3                   Normandale Community College    Unknown
4                                         Unknown    Unknown
5                               Towson University    Unknown
6         New Orleans Baptist Theological Seminary    Unknown
7                           University of Colorado   Bouldero
8                        San Diego State University    Unknown
9  U.S. Army 82nd Airborne Division School of woo...    Unknown
```

The Structure Conversion process was applied to the jobs, currentplace, and previousplace columns as the info in username and userid columns were one value per row. In the jobs column, some rows contained multiple values separated by commas which explains the jobs that the user is currently still active in. to structure the job column, the column was transformed into a list format using the following line code: users['jobs'] = users['jobs'].apply(lambda x: x.split(',') if isinstance(x, str) and pd.notna(x) else [])

users['jobs'] = users['jobs'].apply(lambda x: [job.strip() for job in x] if isinstance(x, list) else x)

This line code helped in splitting comma separated values and removed extra spaces.Additionally, Similar process was applied to curentplace and previousplace columns. Which also helped in restructuring the values separated by commas into a list and the empty values were marked and replaced with Unknown or blank values. Finally, a verifier line code was initiated to ensure the list are well structured and correct.

Spell Checking for users dataset:

```
corrections = {
    'customer Service': 'Customer Service',
    'Uplnd': 'Upland',
    'High Shool': 'High School',
    'Servce': 'Service',
    'Traner': 'Trainer'
}

def quick_correct(lst):
    if isinstance(lst, list):
        return [corrections.get(word.strip(), word.strip()) for word in lst]
    return lst

users['jobs'] = users['jobs'].apply(quick_correct)
users['currentPlace'] = users['currentPlace'].apply(quick_correct)
users['previousPlace'] = users['previousPlace'].apply(quick_correct)
users['education'] = users['education'].apply(quick_correct)

print(users['jobs'].head(10))
print(users[['currentPlace', 'previousPlace', 'education']].head(10))

users.to_csv('cleaned_users.csv', index=False, sep='\t')
```

Spell checking process was done and applied to the users dataset with a manual correction dictionary as multiple errors occurred while downloading the spellchecker package. The purpose of the spell checking process is to enhance the quality of the text type data within the users dataset.

```
0                 [Customer Service]
1                       [technician]
2                                 []
3    [Supplemental Instructon Leader]
4                [Surgical Assistant]
5                      [Lab Monitor]
6          [Church Planter and Pastor]
7                               [CEO]
8                 [Wealth Management]
9                                 []
Name: jobs, dtype: object
```

```
                                currentPlace  \
0                               [Pomona, CA]
1                                         []
2                                         []
3                       [Minneapolis, MN]
4                        [Salt Lake City]
5                                         []
6            [Zion City, Baton Rouge, LA]
7                                         []
8                                 [San Diego]
9  [Hope Mills, NC 28348, United States]
```

	previousPlace	education
0	[Upland, Ca]	Upland High School
1	[Atlanta]	NaN
2	[Detroit, michigan, Bowers, Beach, Delaware, S...	NaN
3	[Honolulu, HI]	Normandale Community College
4	[]	NaN
5	[Houston, TX]	Towson University
6	[Baton Rouge, LA]	New Orleans Baptist Theological Seminary
7	[]	University of Colorado, Bouldero
8	[]	San Diego State University
9	[]	U.S. Army 82nd Airborne Division School of woo...

The results showed the new list with new corrected terms after the use of spell checking process.

```
users.to_csv('cleaned_users_v1.csv', index=False, sep='\t')
print("Users saved as cleaned_users_v1.csv")
```

Finally, the progress has been saved successfully and a new cleaned file has been created and named (cleaned_users_v1.csv) respectively.

Uploading files, structuring, spell checking, and saving the progress for places dataset:

```
]: import pandas as pd
places = pd.read_csv(r'C:\Users\laith\OneDrive\Laith Studying\LSDC-BM Global Assignments\Introduction to data science and big data\DataScienceAssignme
print(places.shape)
print(places.head(10))
places['address'] = places['address'].apply(lambda x: x.split(',') if isinstance(x, str) and pd.notna(x) else [])
places['address'] = places['address'].apply(lambda x: [part.strip() for part in x] if isinstance(x, list) else x)
print(places['address'].head(10))
corrections = {'customer Service': 'Customer Service', 'Uplnd': 'Upland', 'High Shool': 'High School', 'Servce': 'Service', 'Traner': 'Trainer'}
def quick_correct(lst):
    if isinstance(lst, list):
        return [corrections.get(word.strip(), word.strip()) for word in lst]
    return lst
places['address'] = places['address'].apply(quick_correct)
places.to_csv('cleaned_places_v1.csv', index=False, sep='\t')
print("Places saved as cleaned_places_v1.csv")
```

The similar process of the users dataset was done on the places dataset. Which is uploading the file, structuring the unstructured data, spell checking to ensure data quality,and saving the progress with a new cleaned file named (cleaned_places_v1.csv).

Mhd Laith Alkhateeb ID No: 213142 Word Count: 2390

```
(22471, 7)
            gPlusPlaceId                                  name price  \
0  118222137795476771294                           Old Chicago    $$
1  110904357941128849034                       Juliet Photography   NaN
2  113140673306093175878                              McDonald's   $$$
3  113918083535820316316                         The Nucleus Cafe    $$
4  113376774333537973483             Claridge House Apartment Homes   NaN
5  100523848790335586195              die Trauringwelt Fischer e.K.   NaN
6  109867835269515778588  Mountain Oasis International Restaurant    $$
7  110000941779030980175                               Burger King   $$$
8  100316105162546630387                    Sandy's Country Kennels   NaN
9  109588100497504199450                     Stables Downtown Grill    $$
```

The result showed that the file was successfully uploaded. Indicating that the dataset contains 7 columns with approximately 22,471 rows along with showing the first 10 rows as a sample data to verify the process respectively.

```
                address                                  hours  \
0  17960 NW Evergreen Pkwy  Monday 11:00 am--1:00 am Tuesday 11:00 am--1:0...
1      8825 Man Of War Dr                                              NaN
2          399 U.S. 287  Monday Open 24 hours Tuesday Open 24 hours Wed...
3          405 Water St  Monday Closed Tuesday Closed Wednesday 8:00 am...
4          3315 W 4th St  Monday 8:00 am--5:00 pm Tuesday 8:00 am--5:00 ...
5          Karlstraße 28  Monday 11:00 am--7:00 pm Tuesday 11:00 am--7:0...
6          11 E Aspen Ave  Monday 11:00 am--9:00 pm Tuesday 11:00 am--9:0...
7      31456 Woodward Ave  Monday 6:00 am--11:00 pm Tuesday 6:00 am--11:0...
8       123 Stephenson Rd                                              NaN
9         206 N Spring St  Monday 11:00 am--11:00 pm Tuesday 11:00 am--11...
```

```
                phone  closed
3   (503) 533-4650   False
L   (704) 243-3226   False
2   (303) 926-8811   False
3   (715) 834-7777   False
4   (601) 268-8879   False
5    0211 54472626   False
5   (928) 214-9270   False
7              NaN   False
3   (256) 751-1293   False
3   (662) 840-0069   False
3     [17960 NW Evergreen Pkwy]
L           [8825 Man Of War Dr]
2                  [399 U.S. 287]
3                  [405 Water St]
4                [3315 W 4th St]
5                [Karlstraße 28]
5              [11 E Aspen Ave]
7            [31456 Woodward Ave]
3             [123 Stephenson Rd]
3               [206 N Spring St]
Name: address, dtype: object
Places saved as cleaned_places_v1.csv
```

The structuring process showed the new list of columns like address, hours, phone, and closed. And the process was saved as mentioned.

Mhd Laith Alkhateeb ID No: 213142 Word Count: 2390

Uploading files, structuring, spell checking, and saving the progress for reviews dataset:

```
import pandas as pd
reviews = pd.read_csv(r'C:\Users\laith\OneDrive\Laith Studying\L5DC-BM Global Assignments\Introduction to data science and big data
print(reviews.shape)
print(reviews.head(10))
reviews['reviewText'] = reviews['reviewText'].apply(lambda x: x.split(',') if isinstance(x, str) and pd.notna(x) else [])
reviews['reviewText'] = reviews['reviewText'].apply(lambda x: [word.strip() for word in x] if isinstance(x, list) else x)
print(reviews['reviewText'].head(10))
corrections = {
    'customer Service': 'Customer Service',
    'Uplnd': 'Upland',
    'High Shool': 'High School',
    'Servce': 'Service',
    'Traner': 'Trainer'
}
def quick_correct(lst):
    if isinstance(lst, list):
        return [corrections.get(word.strip(), word.strip()) for word in lst]
    return lst
reviews['reviewText'] = reviews['reviewText'].apply(quick_correct)
reviews.to_csv('cleaned_reviews_v1.csv', index=False, sep='\t')
```

```
import pandas as pd
from ast import literal_eval

contraction_dict = {
    "don't": "do not",
    "can't": "cannot",
    "won't": "will not",
    "it's": "it is",
    "you're": "you are",
    "i've": "i have",
    "i'm": "i am",
    "thanks": "thank you"
}
```

```
def expand_contractions(text):
    if isinstance(text, str) and text.startswith('['):
        try:
            text_list = literal_eval(text)
        except (ValueError, SyntaxError):
            return text
        expanded = []
        for phrase in text_list:
            if isinstance(phrase, str) and phrase:
                words = phrase.split()
                expanded_words = [contraction_dict.get(word.lower(), word) for word in words]
                expanded.append(' '.join(expanded_words))
            else:
                expanded.append(phrase)
        return expanded
    elif isinstance(text, str) and text:
        words = text.split()
        return ' '.join([contraction_dict.get(word.lower(), word) for word in words])
    return text
```

```
def tokenize_text(text):
    if isinstance(text, list):
        tokenized = []
        for phrase in text:
            if isinstance(phrase, str) and phrase:
                tokenized.extend(phrase.split())
            else:
                tokenized.append(phrase)
        return tokenized
    elif isinstance(text, str) and text:
        return text.split()
    return text

reviews = pd.read_csv('cleaned_reviews_v1.csv', delimiter='\t')

reviews['reviewText'] = reviews['reviewText'].apply(expand_contractions)
print("After contraction removal:")
print(reviews['reviewText'].head(9))

reviews['reviewText'] = reviews['reviewText'].apply(tokenize_text)
print("After tokenization:")
print(reviews['reviewText'].head(9))

reviews.to_csv('cleaned_reviews_v3.csv', index=False, sep='\t')
print("Reviews saved as cleaned_reviews_v3.csv with contractions removed and tokenized")
```

```
After contraction removal:
0                                                    []
1                                                    []
2    [The guys are dependable, honest and have grea...
3    [As a Minnesota transplant, i have found that ...
4    [I came into Christopher Kia South Jordan look...
5                                                    []
6    [this place is the bomb, i am a meat eater and...
7    [Time Warner Cable is the biggest crap company...
8    [Very Happy with their service, thank you ProS...
Name: reviewText, dtype: object
After tokenization:
0                                                    []
1                                                    []
2    [The, guys, are, dependable, honest, and, have...
3    [As, a, Minnesota, transplant, i, have, found,...
4    [I, came, into, Christopher, Kia, South, Jorda...
5                                                    []
6    [this, place, is, the, bomb, i, am, a, meat, e...
7    [Time, Warner, Cable, is, the, biggest, crap, ...
8    [Very, Happy, with, their, service, thank, you...
Name: reviewText, dtype: object
Reviews saved as cleaned reviews v3.csv with contractions removed and tokenized
```

In the reviews dataset, the loading, structuring, and spell checking were completed along with adding contraction removal and the reviews column has been tokenized . all by using manual dictionaries due to incapabilities of installing packages. Finally , the progress has been saved and named (cleaned_reviews_v3).

Task 2: Data Processing

This task explores the performance of data processing on the cleaned datasets by addressing certain questions.

a) Retrieving all places that have more than 3 reviews.

```
review_counts = reviews.groupby('gPlusPlaceId').size().reset_index(name='#reviews')

places_with_many_reviews = review_counts[review_counts['#reviews'] > 3]

places_with_many_reviews = places_with_many_reviews.sort_values('#reviews', ascending=False)

print(places_with_many_reviews[['#reviews', 'gPlusPlaceId']])
```

```
       #reviews           gPlusPlaceId
1584          4  100515652515321866651
5224          4  102250986789579477120
6628          4  103029351909058756043
10109         4  105590614995402411627
```

The results showed every place that has got more than 3 reviews. The technique used to retrieve the desired data is (groupby) which is considered a powerful technique that highly recommended to split and group dataframes and can work on large datasets. (geeksforgeeks.org , Pandas dataframe.groupby() Method, 11 jul, 2025)

b) Compute the number of places in the Places dataset that are permanently closed

```
closed_counts = places.groupby('closed').size().reset_index(name='#places')

closed_counts = closed_counts[['#places', 'closed']].rename(columns={'closed': 'is_closed'})

print(closed_counts)
```

```
   #places  is_closed
0    21715      False
1      756       True
```

The results showed the two columns which represent the places that are still operating and the places that are permanently closed. Moreover, the places that are still operating are 21,715. While the places that are permanently closed are valued at 756 places.

(groupby) technique is used in this process along with size() function as it is efficient with grouping and counting . (geeksforgeeks.org , Grouping and Aggregating with Pandas, 26 Jul, 2025).

c) Displaying the 20 most frequent phrases that appear in the review Text of reviews that have a rating of 5.

```
from collections import Counter
from itertools import chain

high_reviews = reviews[reviews['rating'] == 5.0]

phrases = []
for text in high_reviews['reviewText']:
    if isinstance(text, list):
        words = text  # Assuming tokenized list
    else:
        words = text.split()
    for i in range(len(words) - 2):
        phrase = ' '.join(words[i:i+3])
        phrases.append(phrase)

phrase_counts = Counter(phrases).most_common(20)

print("Rating: 5.0")
print('; '.join(f"{phrase} {count}" for phrase, count in phrase_counts))
```

```
Rating: 5.0
'I', 'highly', 'recommend', 314; 'one', 'of', 'the', 268; 'I', 'would', 'recommend', 213; 'I', 'have', 'been', 196; 'of', 'the', 'best', 185; 'a', 'lo
t', 'of', 180; 'looking', 'for', 'a', 176; 'would', 'highly', 'recommend', 176; 'is', 'a', 'great', 175; 'I', 'would', 'highly', 166; 'is', 'the', 'bes
t', 157; ['I', 'have', 'been', 153; 'I', 'have', 'ever', 151; 'was', 'able', 'to', 146; 'it', 'is', 'a', 145; 'as', 'well', 'as', 141; 'and', 'it', 'wa
s', 135; 'The', 'staff', 'is', 133; 'is', 'one', 'of', 132; 'very', 'friendly', 'and', 128
```

The results of the provided execution represent the most frequent phrases that had been written by reviewers who gave 5 stars to the places they had visited . For example , in the provided execution, the phrase (I highly recommend) was written 314 times by different users who gave a rate of 5 to the places they had visited across the dataset. The techniques used in this process are (Counter) for frequency (realpython.com , Python's Counter: The Pythonic Way to Count object) and (join) for phrases (simplilearm.com , Join in Python: Definition, Usage, Syntax, and Parameters , Jan 26, 2025).

The Counter function can count elements in a list efficiently (geeksforgeeks.org , Python Collections Counter, 23 Jul, 2025), and the join function can accurately create phrases (hyperskill.org , join() Method in Python , Oct 3, 2024).

d) Creating a subset of the user's dataset by keeping only the 'IT Specialist' users

```
it_specialists = users[users['jobs'].apply(lambda x: 'IT Specialist' in x if isinstance(x, list) else False)]

print(it_specialists)

it_specialists.to_csv('it_specialists_subset.csv', index=False, sep='\t')
print("Subset saved as it_specialists_subset.csv")
```

```
                gPlusUserId        userName             jobs  \
71     100003205829319729266  Philip Chapman  [IT Specialist]
4403   100232638021775806638    Wayne Millen  [IT Specialist]
9846   100525974332808520587       Quang Vũ  [IT Specialist]
13966  100747265098330634882    Colin Tracey  [IT Specialist]
17021  100913985877815378529      Corey Webb  [IT Specialist]
```

The results of the execution provided is a subset of the cleaned users datasets that keeps the (IT Specialist) only. The results also indicated that only 5 users in the whole dataset hold the position of IT specialist and have been merged into a single subset. The technique used in this process is (apply) as it can deal with columns in the dataframes with different types of data such as the list columns. (medium.com , Python- (Cleaning Data With Pandas): Part 9.Using Apply and Map Functions in Pandas, May 7, 2025)

e) Finding the place (id and name) that has received the largest number of reviews.

```
review_counts = reviews.groupby('gPlusPlaceId').size().reset_index(name='#reviews')
merged = pd.merge(review_counts, places[['gPlusPlaceId', 'name']], on='gPlusPlaceId')
top_place = merged.loc[merged['#reviews'].idxmax()]
print(f"#reviews {top_place['#reviews']} {top_place['gPlusPlaceId']} {top_place['name']}")
```

```
#reviews 4 102250986789579477120 Scott Clark Honda
```

The result of the execution provided is the place that has received the largest number of reviews across the dataset along with the name of the place (Scott Clark Honda). The place appears to be one of the places that have more than 3 reviews as per the results of part (a). This happens due to the place's popularity and people wanted to share their experience about this particular place.The

techniques used in this process are (groupby) as it is efficient in counting and grouping as discussed and (merge) name as it can link and merge datasets accurately.

(geeksforgeeks.org , Merge/Join Two Dataframes on Multiple Columns in Pandas, 23 Jul, 2025)

Task 3: Data Analysis

This task analyzes the data from the cleaned reviews dataset by using and justifying suitable techniques

a) Displaying the number of positive, neutral and negative reviews using a suitable technique.

```
import pandas as pd
reviews = pd.read_csv('cleaned_reviews_v3.csv', delimiter='\t')
print(reviews.shape)
print(reviews.head())
```

```
(21746, 7)
            gPlusPlaceId            gPlusUserId  rating      reviewerName  \
0  104616184831707794985  100000035085750632094     5.0  Jacquelyn Dorris
1  117478827630848991185  100000277413564180184     3.0     Pavel Sysuyeu
2  103005817928390585722  100000283953302773884     4.0    Linda Campbell
3  111258782406510596618  100000328262218933119     5.0        Adam Perry
4  101843268008601766841  100000356948930389974     5.0   Courtney Rogers

                                        reviewText        categories  \
0                                               []           College
1                                               []               Gym
2  ['The', 'guys', 'are', 'dependable', 'honest',...  Auto Repair Shop
3  ['As', 'a', 'Minnesota', 'transplant', 'i', 'h...   Animal Hospital
4  ['I', 'came', 'into', 'Christopher', 'Kia', 'S...        Kia Dealer

     reviewTime
0  Mar 16, 2012
1  Feb 16, 2013
2  Feb 12, 2014
3  Aug 29, 2013
4  Jun 21, 2013
```

In order to proceed in the process of extracting the reviews, the cleaned review file was loaded to specify the columns needed.

```python
def classify_by_rating(rating):
    if rating > 3:
        return 'Positive'
    elif rating < 3:
        return 'Negative'
    else:
        return 'Neutral'

reviews['sentiment'] = reviews['rating'].apply(classify_by_rating)
sentiment_counts = reviews['sentiment'].value_counts().reset_index()
sentiment_counts.columns = ['Sentiment', '#Reviews']
print(sentiment_counts)
```

```
  Sentiment  #Reviews
0  Positive     17160
1  Negative      2750
2   Neutral      1836
```

After successfully loading the file, the rating column is used to identify the reviews based on the following rules:

1- if the rating was >3, then, it is positive

2- if the rating was <3, then, it is negative

3- if the rating was =3, then, it is neutral

The results showed the number of reviews that each review type has achieved. 17160 reviews were marked positive, 2750 were marked negative, and 1836 reviews were marked neutral.

b) Justifying selected techniques

The technique used in calculating the positive, negative, and neutral reviews is a simple rule based on the rating columns.indicating that the ratings greater than 3 are considered positive, less than 3 are negative, and equal to 3 are neutral. The rating based evaluation technique is a straightforward technique that is less time consuming and effective. It helps in eliminating the need of installing specific packages when errors could be a major challenge. Additionally, tools like VADER is the first strong option to consider when doing such analysis as it is capable of reading texts that include informal language, emotional tones, and emojis to calculate scores. (geeksforgeeks.org , Sentiment Analysis using VADER - Using Python, 11 Jul, 2025).

Task 4

Based on the insights, the given ratings showed how satisfied the customer is. For example, the largest number of feedback appears to be positive. With a total of 17160 positive reviews. Which is a great sign that the most of the customers were happy with the services given by places. Phrases like (I highly recommend, 314 appearances) and (I would recommend, 213 appearances) show a solid approval by customers. This solid approval by customers indicated through these phrases can help location owners seek strong advertising and commercial campaigns solutions to elevate their reach, and show capabilities of delivering the best customer service consistently. Which can lead to increasing the customers' trust and loyalty to the brand. Additionally, the phrase (looking for, 176 appearances) indicates that the customer is seeking for certain services which he/she could not find in any other place but found it in places given in the dataset. This phrase can help location owners to understand the market needs and what the customer is looking for, making them outstand any other similar brand that delivers similar services by offering the opportunity of giving services that customers seek to have which they could not find in rival brands. Leading to the increase of the revenues of the brand. Furthermore, phrases such as (the staff is, 133 appearances) and (very friendly, 128 appearances) indicate the existence of the cooperation and the friendly behavior of the staff in most of the locations in the given dataset. These phrases can help the location owners to motivate their staff and show care for them by initiating more training sessions focusing on delivering exceptional customer experience. These phrases also emphasize the need for a friendly relationship between the owners and the staff members. Which can lead to increasing the loyalty of the staff for the brand and always motivate them to do the best they could. Moreover, reviews with lower ratings might indicate minor issues like wait time. These issues can be addressed by generating monthly reports to find out the strengths and weaknesses, and keeping in touch with staff members to discover whether an operational review could be considered.

By adhering to the analysis, the chance of receiving neutral or negative ratings and reviews will significantly decrease. Leading to the consistency of delivering the best customer experience and increasing customer satisfaction. However, the safety of users' data should always be a top priority to any entity to build confidence with the user. When users know their data is safe and is not threatened with a potential breach, the chance of engagement will increase. Maintaining user trust and compliance with regulations such as GDPR is a must . according to proofpoint.com article talking about what GDPR is ,

> With GDPR compliance, the level of any personal data processing must be proportional to the purpose of collecting it. That means collecting as little data as possible and keeping it no longer than necessary to serve the customer.
>
> You must keep the data accurate and up to date. And you must protect its confidentiality and integrity. GDPR policies require organizations to bake **data protection** into their infrastructure design by default and establish a proactive approach to safeguard consumer information.